Software Requirements Specification

for

Team Project Management Platform: Collabra

Version 1.0 approved

Prepared by Group 8
Jason Alvaro Gouw, Lintang Anggowoyuono, Muhammad Nizwa

BINUS University

25 February 2026

Table of Contents

Revision History

Name	Date	Reason For Changes	Version
Group 8	25-02-2026	Progress Chapter 1	1.0
Group 7	04-03-2026	Progress Chapter 2 Making initial progress for Chapter 2.1 based on requirements in Chapter 1 from Group 8	2.0
Group 7	01-04-2026	Progress Chapter 2 Making UML diagrams	2.1
Group 7	07-04-2026	Progress Chapter 2 Finalizing Use Case Diagram and completion of chapter 2.2	2.2
Group 7	21-04-2026	Progress Chapter 2 Finalizing Activity Diagram and Sequence Diagram and completion of Chapter 2.3 - 2.7	2.3
Group 7	22-04-2026	Progress Chapter 2 Insertion of UML diagrams into SRS	2.4
Group 7	23-04-2026	Progress Chapter 3 Finalizing Figma Design and Completion of Chapter 3	3.0
Group 7	29-04-2026	Progress Chapter 3 Insertion of Figma design into SRS	3.1

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to define in detail the software requirements for the team project management system. This document covers the entire scope of the core application system, focusing on task management, team collaboration, scheduling, and project progress reporting. This SRS will serve as the primary reference, written agreement, and technical guide for developers, quality assurance (QA) testers, and stakeholders throughout the software development life cycle.

1.2 Document Conventions

This document is structured using the IEEE standard with specific typographical conventions to facilitate reading. Bold text is used to highlight important terms, system actors, or user interface (UI) elements, while italic text is used for special emphasis, references to other documents or web pages, and foreign terms. Furthermore, priorities assigned to high-level requirements are assumed to be automatically inherited by all detailed sub-requirements beneath them, unless it is explicitly stated otherwise

1.3 Intended Audience and Reading Suggestions

This SRS document is intended for various parties involved in the development and evaluation of Team Project management Platform. The following are reading suggestions based on specific roles:

1) **Project Manager** (SONYA RAPINTA MANALU, S.Kom., M.T.I.) : Advised to focus on the Introduction, Product Scope, and Overall Description sections to ensure the system being built aligns with the established project objectives and academic requirements.
2) **Software Developers** (Group 8) : Must read the entire document thoroughly, with a primary focus on the System Features and Specific Requirements sections to serve as the main technical guideline for coding and building the standard web application.

1.4 Product Scope

The Team Project Management Platform is a comprehensive web-based application designed to streamline project workflows, optimize resource allocation, and significantly enhance collaboration among team members. The system acts as a centralized workspace and a "Single Source of Truth" tailored for modern teams, allowing both project managers and members to operate efficiently within a unified environment. The core objective of the application is to provide a robust suite of features, which includes:

1) Task Management: Dynamically create, assign, establish priority levels, and set clear deadlines for project tasks.
2) Real-Time Progress Tracking: Monitor individual contributions and collective project milestones using visual status boards.

3) File Sharing & Collaboration: Seamlessly attach project-related files and centralize communication to maintain context within specific tasks.
4) Role-Based Access Control: Differentiate permissions between project managers and regular team members to ensure data integrity and secure delegation of authority.
5) Reporting Dashboard : View overall project health, task distribution, and timeline progress through interactive visual dashboards.

1.5 References

<List any other documents or Web addresses to which this SRS refers. These may include user interface style guides, contracts, standards, system requirements specifications, use case documents, or a vision and scope document. Provide enough information so that the reader could access a copy of each reference, including title, author, version number, date, and source or location.>

2. Overall Description

2.1 Product Perspective

<Describe the context and origin of the product being specified in this SRS. For example, state whether this product is a follow-on member of a product family, a replacement for certain existing systems, or a new, self-contained product. If the SRS defines a component of a larger system, relate the requirements of the larger system to the functionality of this software and identify interfaces between the two. A simple diagram that shows the major components of the overall system, subsystem interconnections, and external interfaces can be helpful.>

The Team Project Management Platform: Collabra will be a new system developed for collaborative and small-team project environments. The system will be built as a standalone web-based system. As it is a newly developed standalone system, it does not serve as a follow-on product or a replacement of an existing commercial system. Instead, it is designed as a self-contained application intended to independently support academic and small-team project collaboration needs.

At this stage of development, Collabra is not designed to integrate with external productivity tools, enterprise systems, or third-party platforms. Features such as synchronization with external calendars, integration with cloud storage services, or connectivity with communication platforms are not being considered within the scope of this version. The system is intended to function independently without reliance on external subsystems.

The primary external interfaces of the system are limited to:

1) Web browsers used by end users (e.g., Chrome, Edge, Firefox)
2) A backend database system responsible for storing user, project, task, and reporting data
3) Local or server-based file storage mechanisms for uploaded documents

The system environment consists of three major components:

1) Client Layer : Web-based user interface accessed by project managers and team members.
2) Application Layer : Server-side logic handling authentication, task processing, and reporting functions.
3) Data Layer : Database responsible for persistent data storage and retrieval.

The following context diagram illustrates Collabra's position as a standalone system and its interaction with users and internal system components.

2.2 Product Functions

The Collabra platform's functionality is centered around providing a centralized workspace for project oversight and team execution. The following Use Case Diagram illustrates the functional boundaries of the system and the interactions between the Project Manager and Team Member roles.

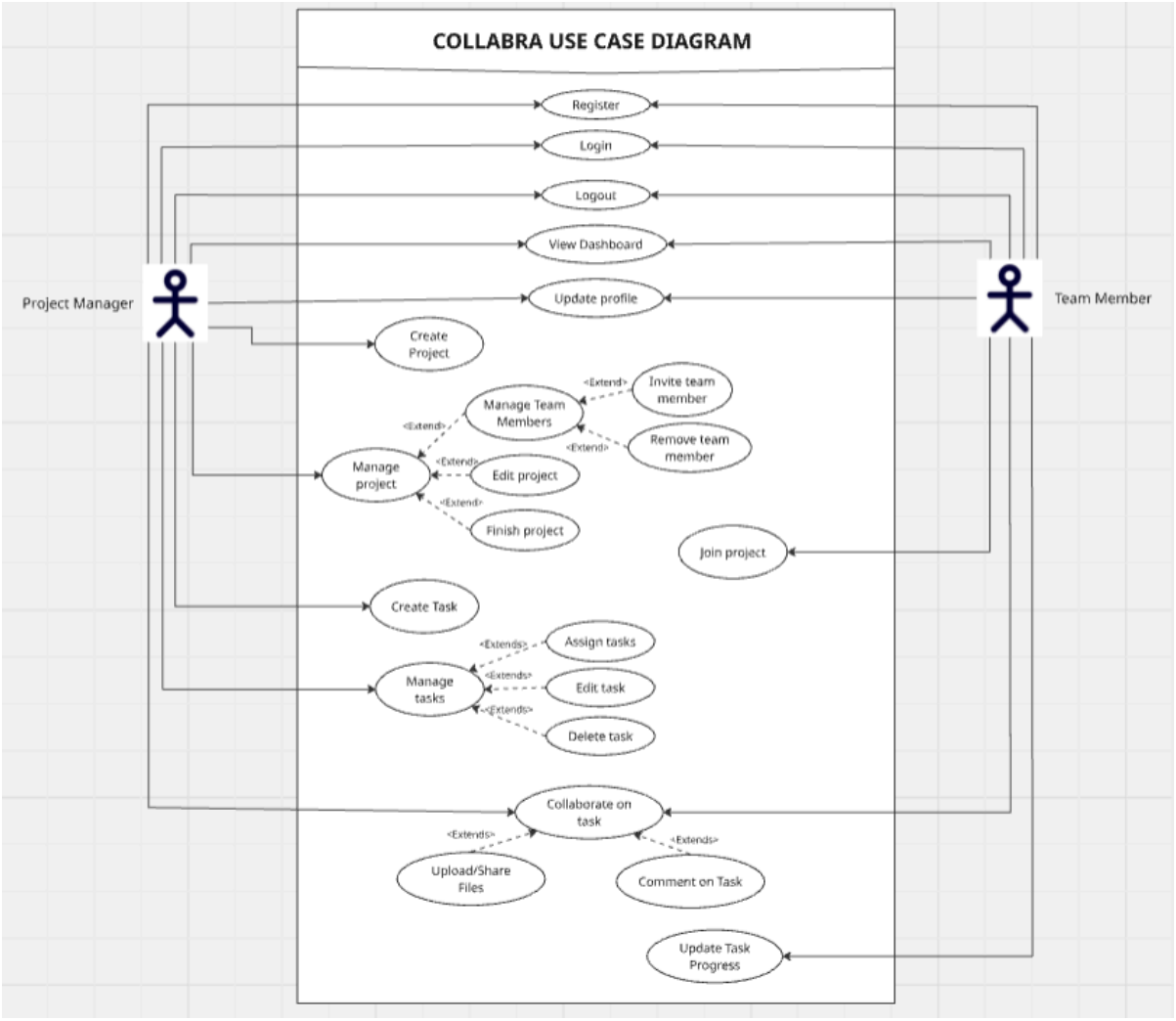


Figure 2.1: Collabra System Use Case Diagram

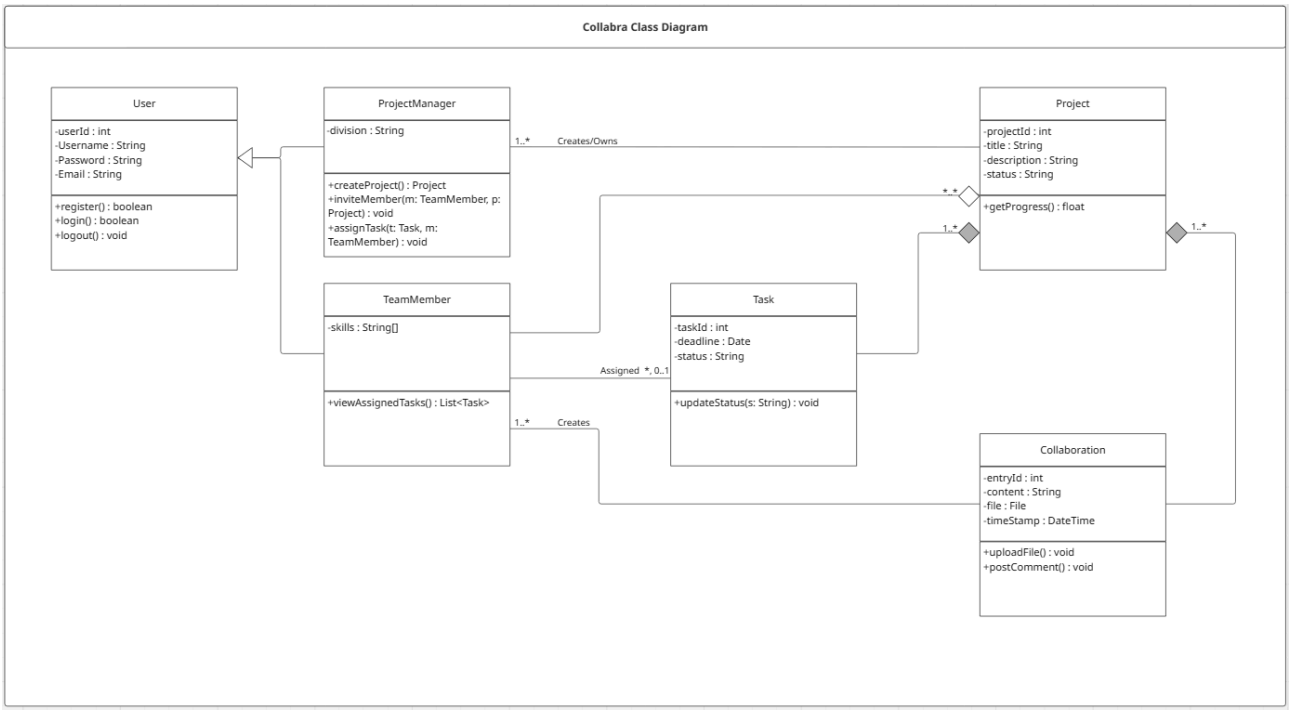


Figure 2.2: Collabra System Class Diagram

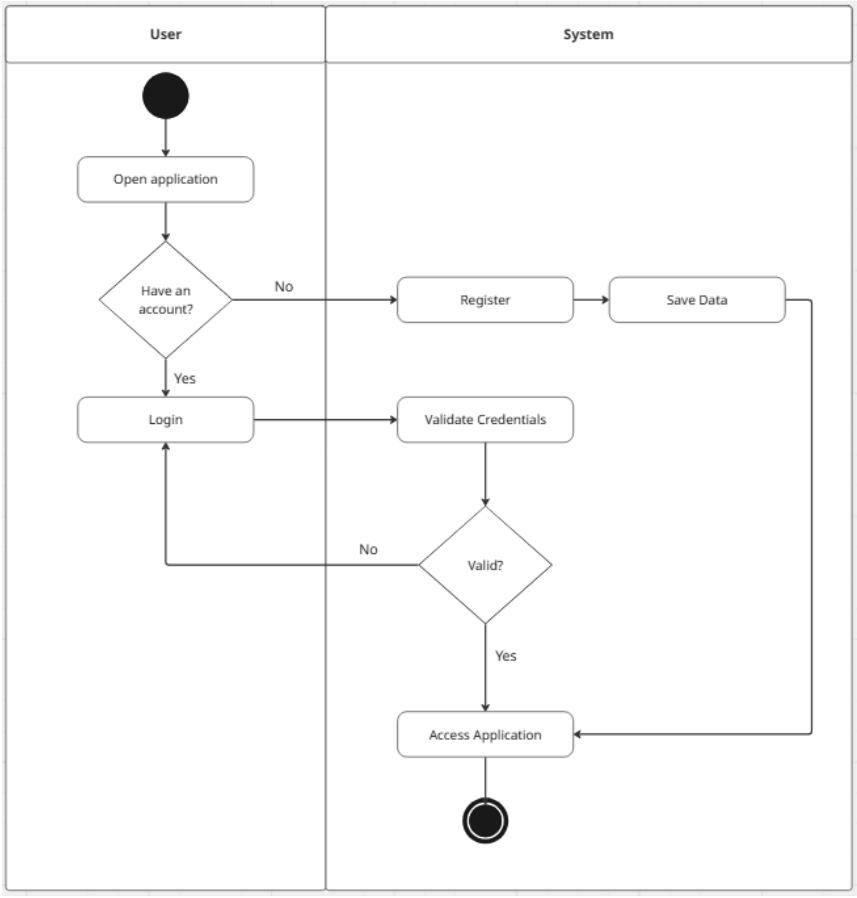


Figure 2.3: Collabra Account Creation and Verification Activity Diagram

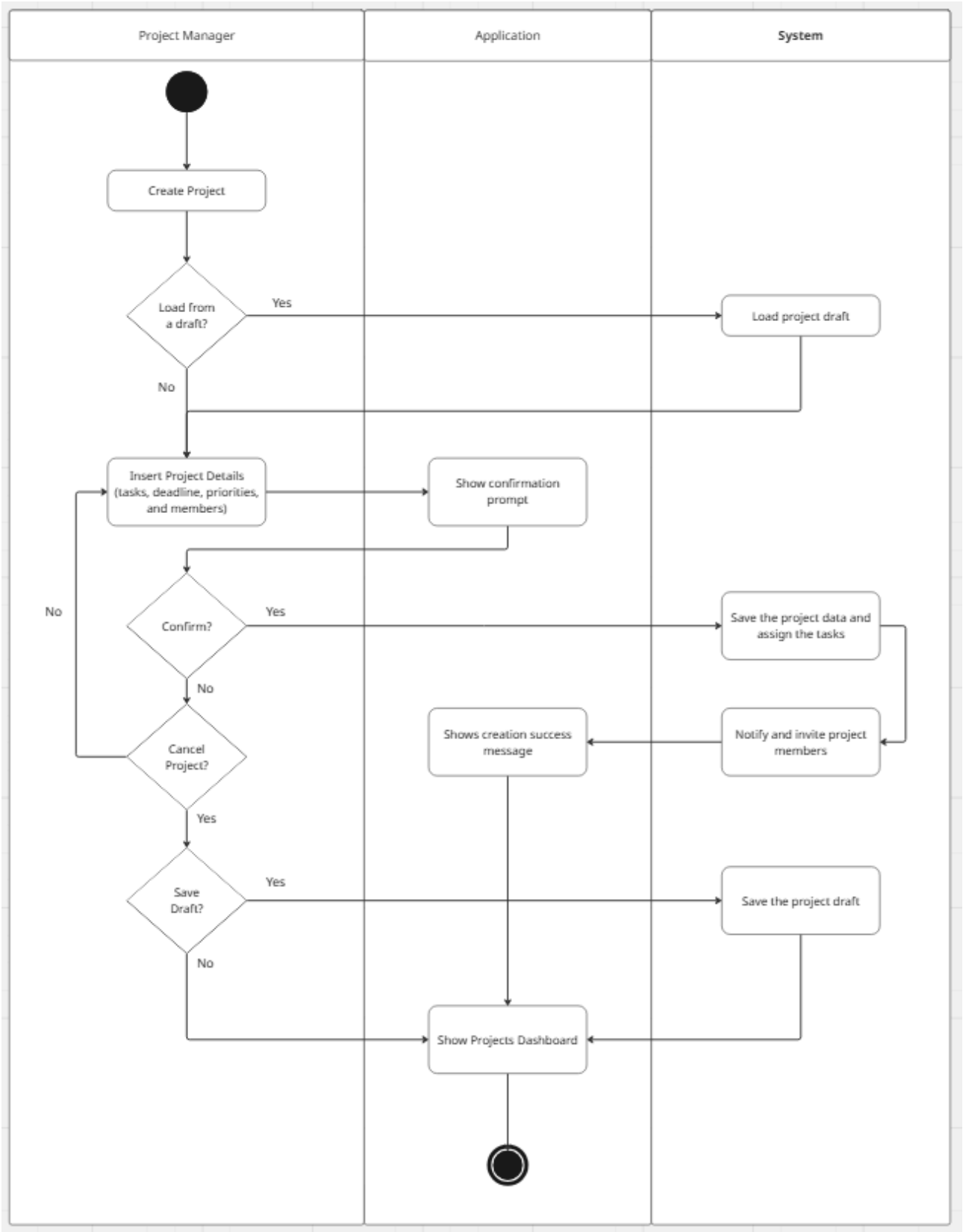


Figure 2.4: Collabra Project Creation Activity Diagram

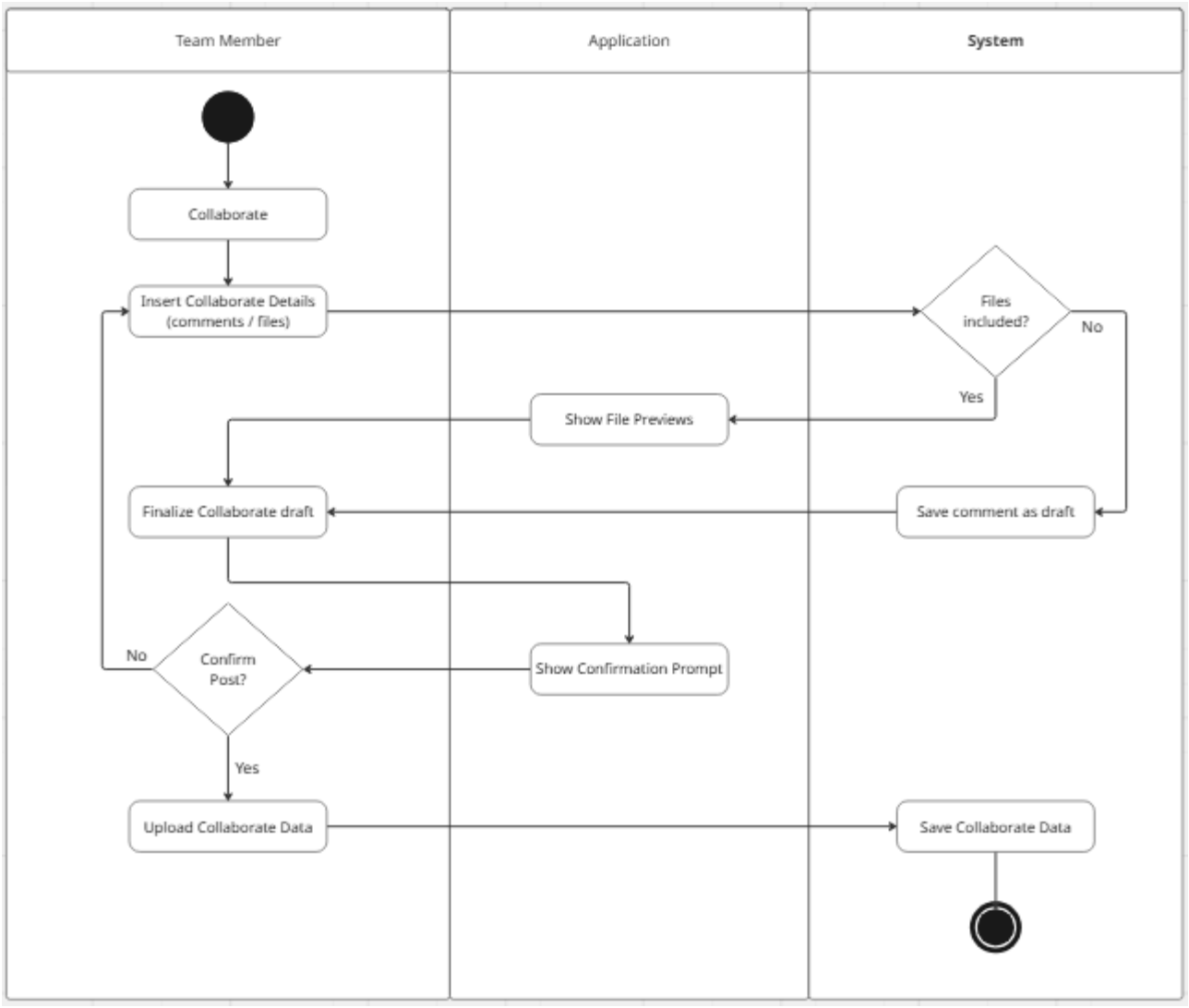


Figure 2.5: Collabra's "Collaborate" Feature Activity Diagram

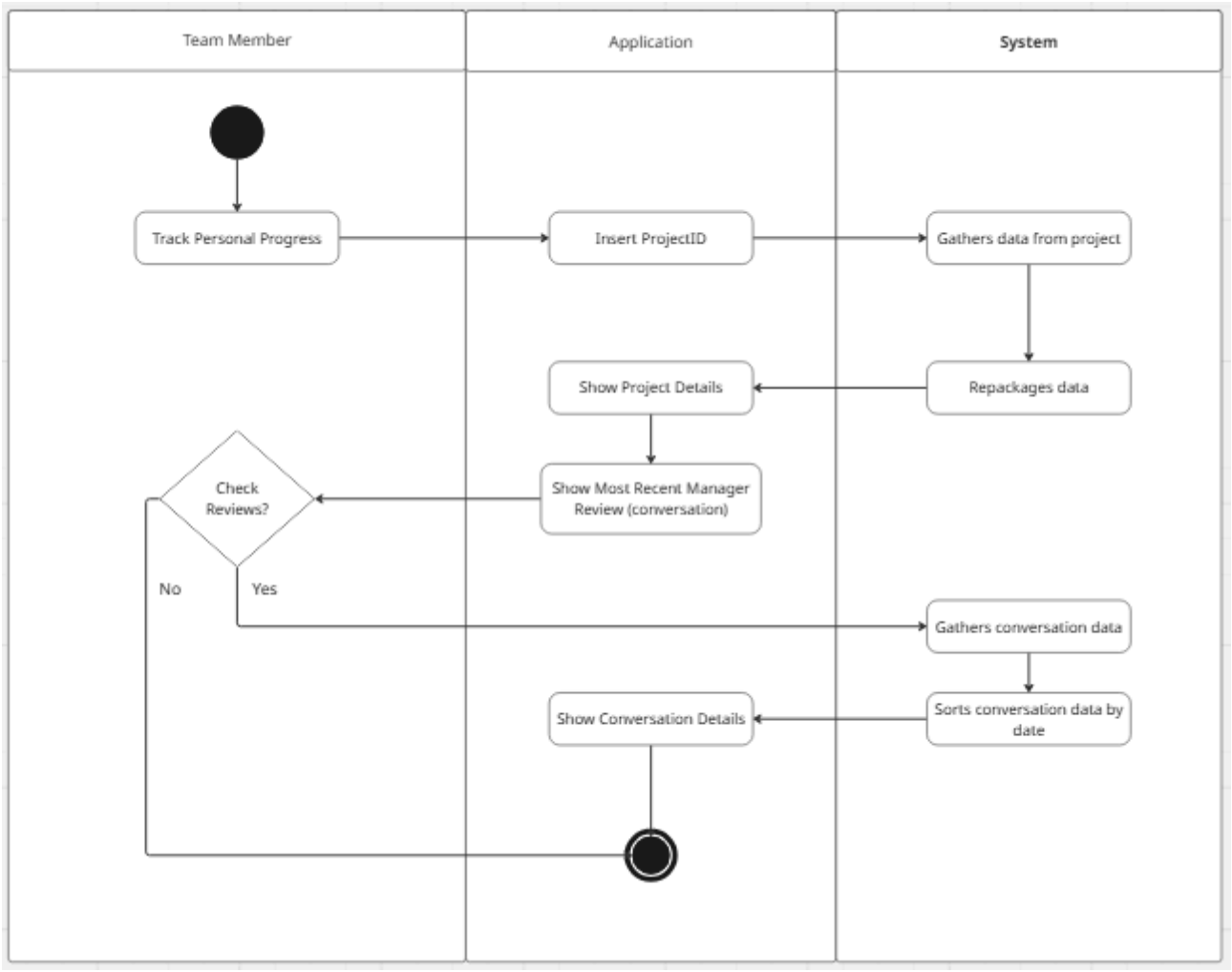


Figure 2.6: Collabra Member Progress Activity Diagram

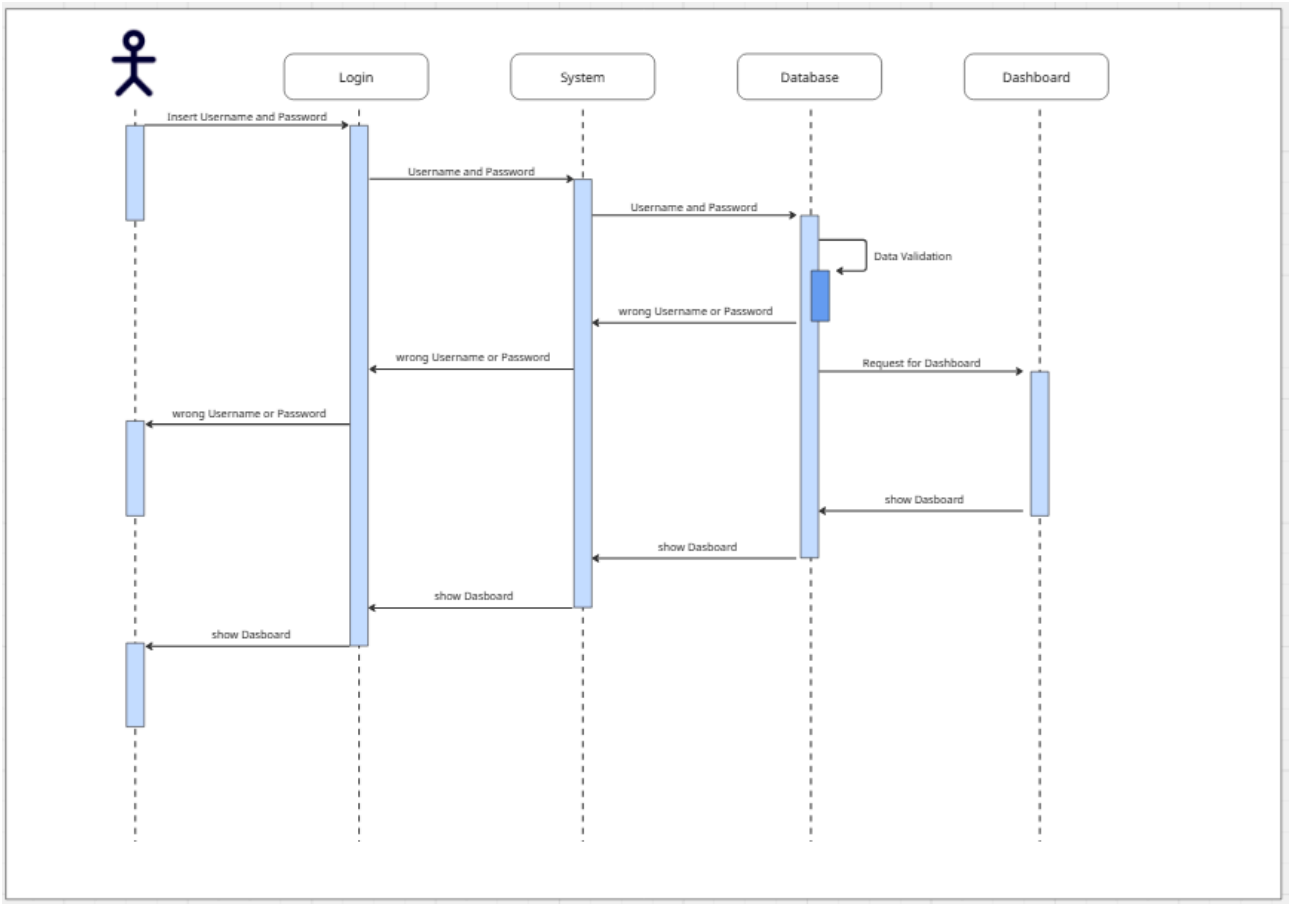


Figure 2.7: Collabra Show Dashboard Sequence Diagram

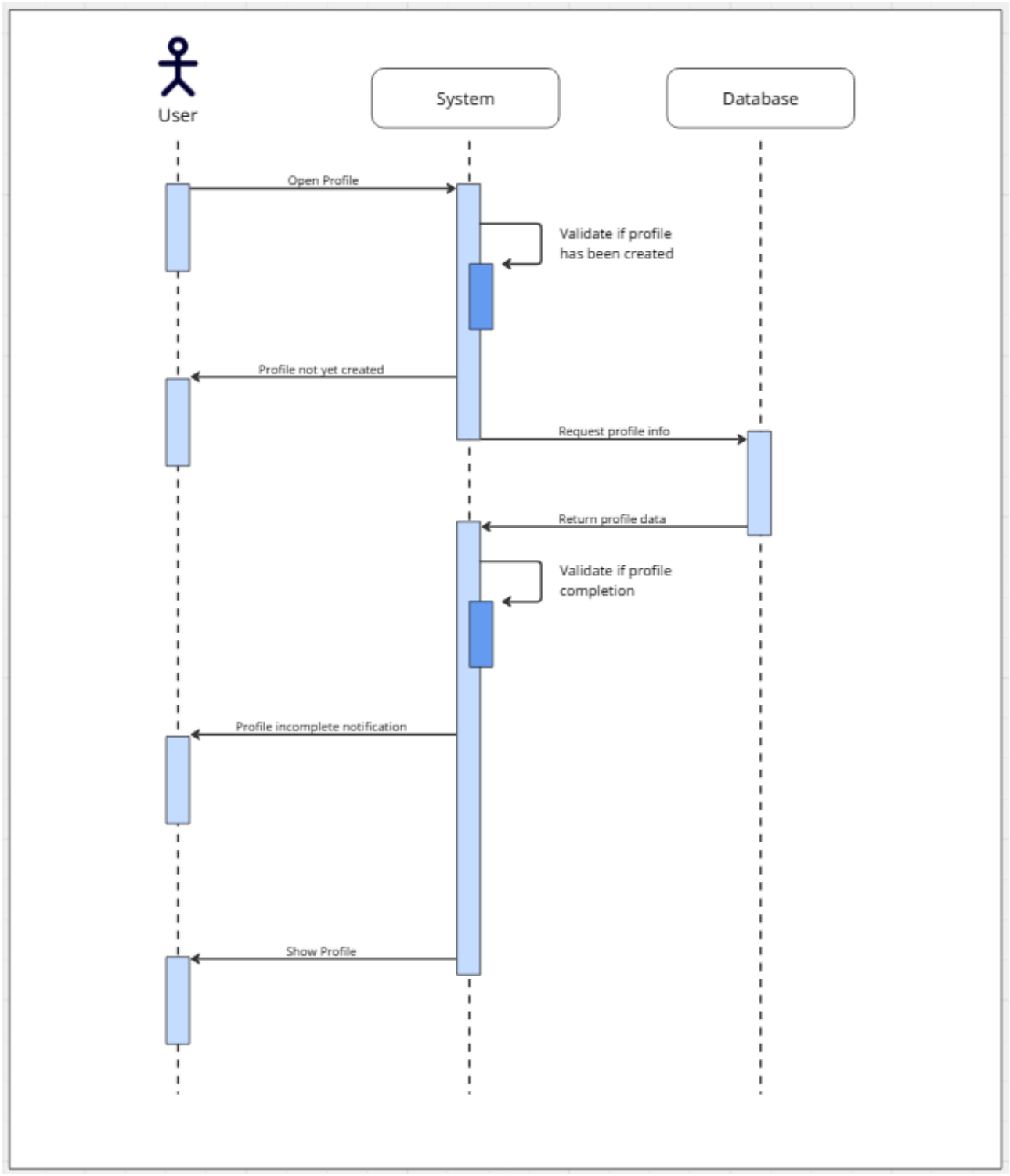


Figure 2.8: Collabra User Profile Completion Check Sequence Diagram

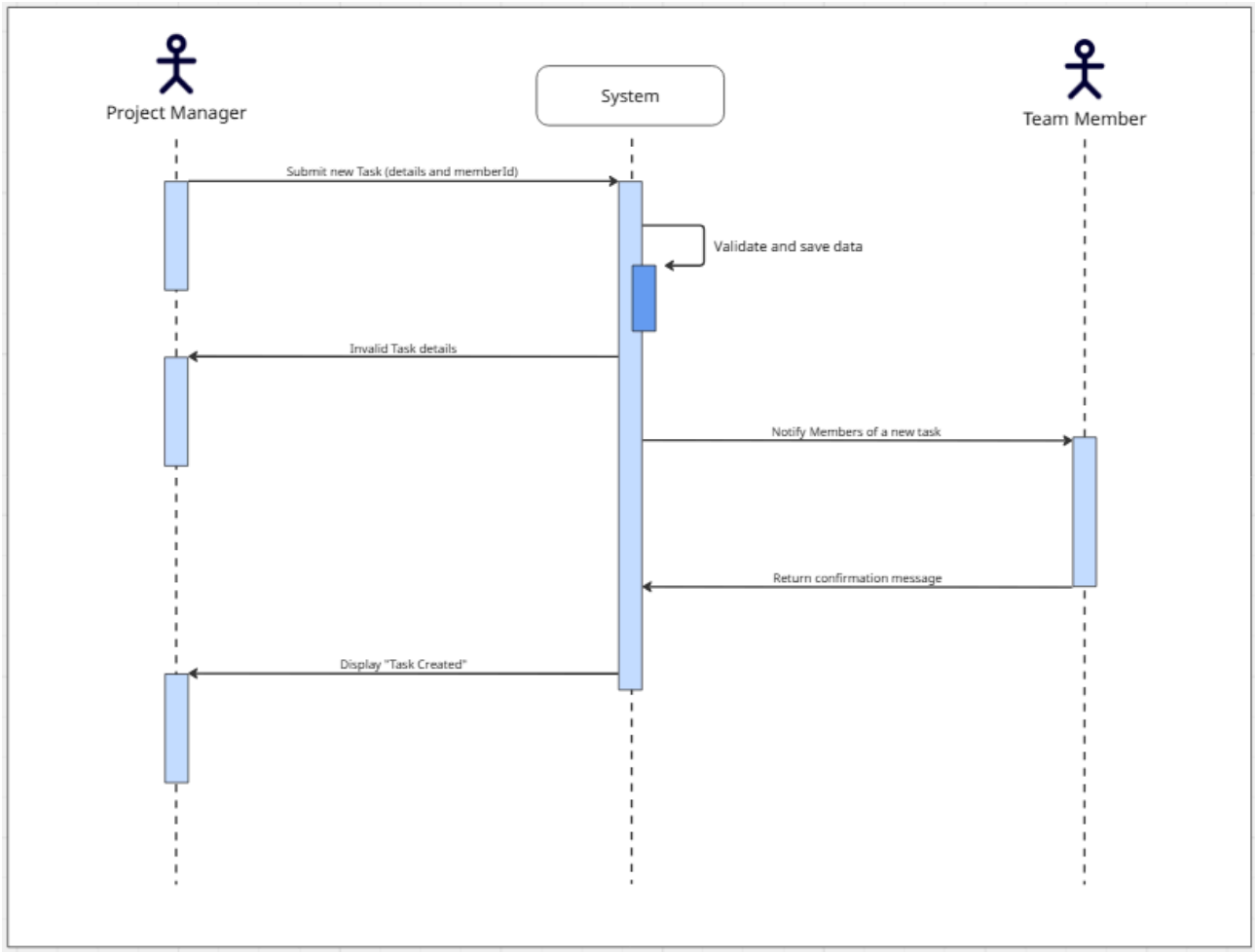


Figure 2.9: Collabra Task Creation Sequence Diagram

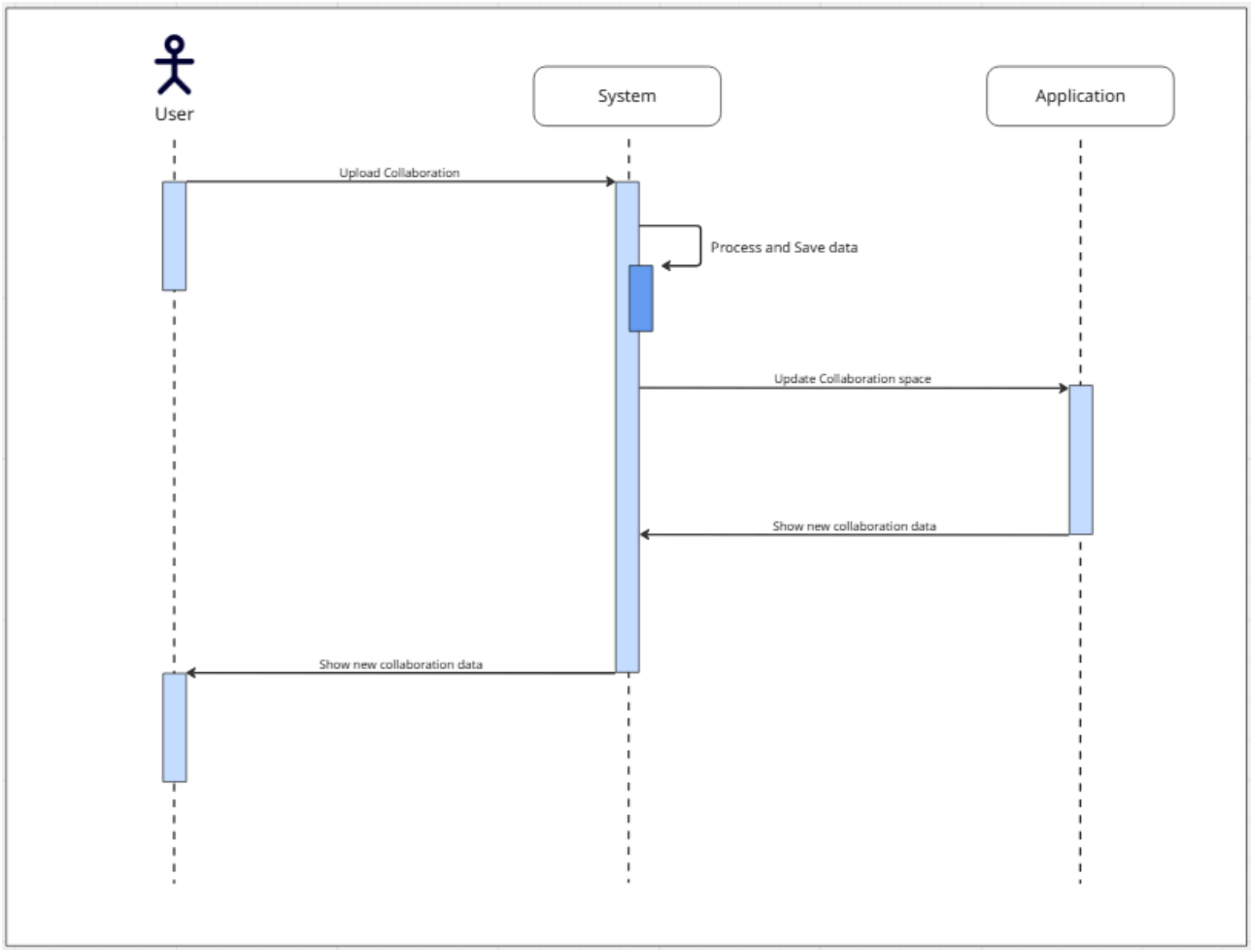


Figure 2.10: Collabra's "Collaborate" Feature Sequence Diagram

The Collabra platform will provide the following major functions, as shown in the Figures 2.1 - 2.10:

1) User Authentication and Profile Management
    a) Register: Allows new users to create an account within the system.
    b) Login/Logout: Provides secure access control for authenticated sessions.
    c) Update Profile: Enables users to manage personal information and account settings.

2) Project Governance
    a) Create Project: Enables the Project Manager to initialize a new project workspace.
    b) Manage Project: A centralized administrative hub for project oversight, which includes:
        i) Invite Team Member (Extend): Optionally adding users to the project roster.
        ii) Remove Team Member (Extend): Optionally removing users from the project.
        iii) Finish Project (Extend): Marking a project as completed or archived.
        iv) Join Project: Allows a Team Member to enter a project workspace they have been assigned to.

3) Task Orchestration
    a) Create Task: Allows the Project Manager to define new work items.
    b) Manage Tasks: The primary interface for task lifecycle management, including:
        i) Assign Tasks (Extend): Linking specific team members to work items.
        ii) Edit Task (Extend): Modifying task attributes such as descriptions or deadlines.
        iii) Delete Task (Extend): Removing a task permanently from the project record.

4) Team Collaboration & Progress Tracking
    a) Collaborate on Task: The functional workspace for task-level interaction, which includes:
        i) Upload/Share Files (Extend): Attaching relevant documents to a specific task.
        ii) Comment on Task (Extend): Facilitating threaded discussions within a task context.
    b) Update Task Progress: Enables Team Members to change the status of their assigned work (e.g., from "In Progress" to "Done").
    c) View Dashboard: Provides a real-time visual overview of project health and milestones.

2.3 User Classes and Characteristics

The platform will serve the following user classes:

1) Project Managers
    a) Team leads or academic supervisors responsible for project outcomes.
    b) Needs: Oversight of all team activities, authority to manage the roster, and ability to delete or reassign tasks.

2) Team Members
    a) Developers, researchers, or contributors.

b) Needs: Clear visibility of assigned tasks, ability to provide progress updates, and a centralized place for discussion.

All users are expected to have basic proficiency with web browsers, though no advanced project management certification is required as Collabra is intended to be accessible for small-scale projects too.

2.4 Operating Environment

The platform will operate in the following environment:

1) Technical Environment
    a) Web-based application accessible via standard modern browsers (Chrome, Safari, Firefox).
    b) Responsive design for mobile-friendly status checking.

2) Hardware Environment
    a) Cloud-hosted server infrastructure with high-availability.
    b) Persistent storage for transaction history and user-uploaded media.

3) Software Environment
    a) Modern web frameworks (e.g., React.js for Frontend, Node.js or Python for Backend).
    b) Relational database management system (RDBMS) for data integrity.

2.5 Design and Implementation Constraints

The following constraints will impact the design and implementation of the platform:

1) Technical Constraints
    a) Must maintain a "Single Source of Truth" by preventing data duplication.
    b) Must function with low latency even as the task database grows.

2) Regulatory Constraints
    a) Must comply with standard data protection principles (encryption of user passwords).
    b) Must adhere to secure file-sharing protocols.

3) Business Constraints
    a) The system must be intuitive enough for non-technical members to use without extensive training.

2.6 User Documentation

The following user documentation will be developed as part of the system:

1) Online Help

a) FAQ section regarding project visibility and task permissions.

2) User Manuals
    a) "Project Manager's Guide" for project setup
    b) "Member's Guide" for task execution.

2.7 Assumptions and Dependencies

Assumptions

1) Users have consistent internet access for real-time updates.
2) Team members will proactively update task statuses to ensure data accuracy.

Dependencies

1) Availability of third-party SMTP servers for invitations.
2) Continuity of cloud storage services for file persistence.

3. External Interface Requirements

3.1 User Interfaces

Collabra provides a web-based user interface intended for two main user classes, namely Project Managers and Team Members, as defined in Chapter 2. The interface is designed to support the main system functions, including authentication, project management, task coordination, team collaboration, file sharing, reporting, and profile management in a centralized workspace.

Based on the approved Figma design, the user interface includes several main screens such as Create Account, Sign In, Dashboard, Projects, Tasks, Team, Files, Reports, Settings, Join Project, Create Task, Create Project, and Task Discussion / Progress Update. These screens reflect the functional scope described in Chapters 1 and 2.

The interface follows a consistent layout across authenticated pages. Most screens use a left-side navigation panel containing the main modules (Dashboard, Projects, Tasks, Team, Files, Reports, Settings) and a top bar containing search and account-related controls. This consistency helps users move between modules efficiently and supports the requirement that the system must remain intuitive for non-technical users.

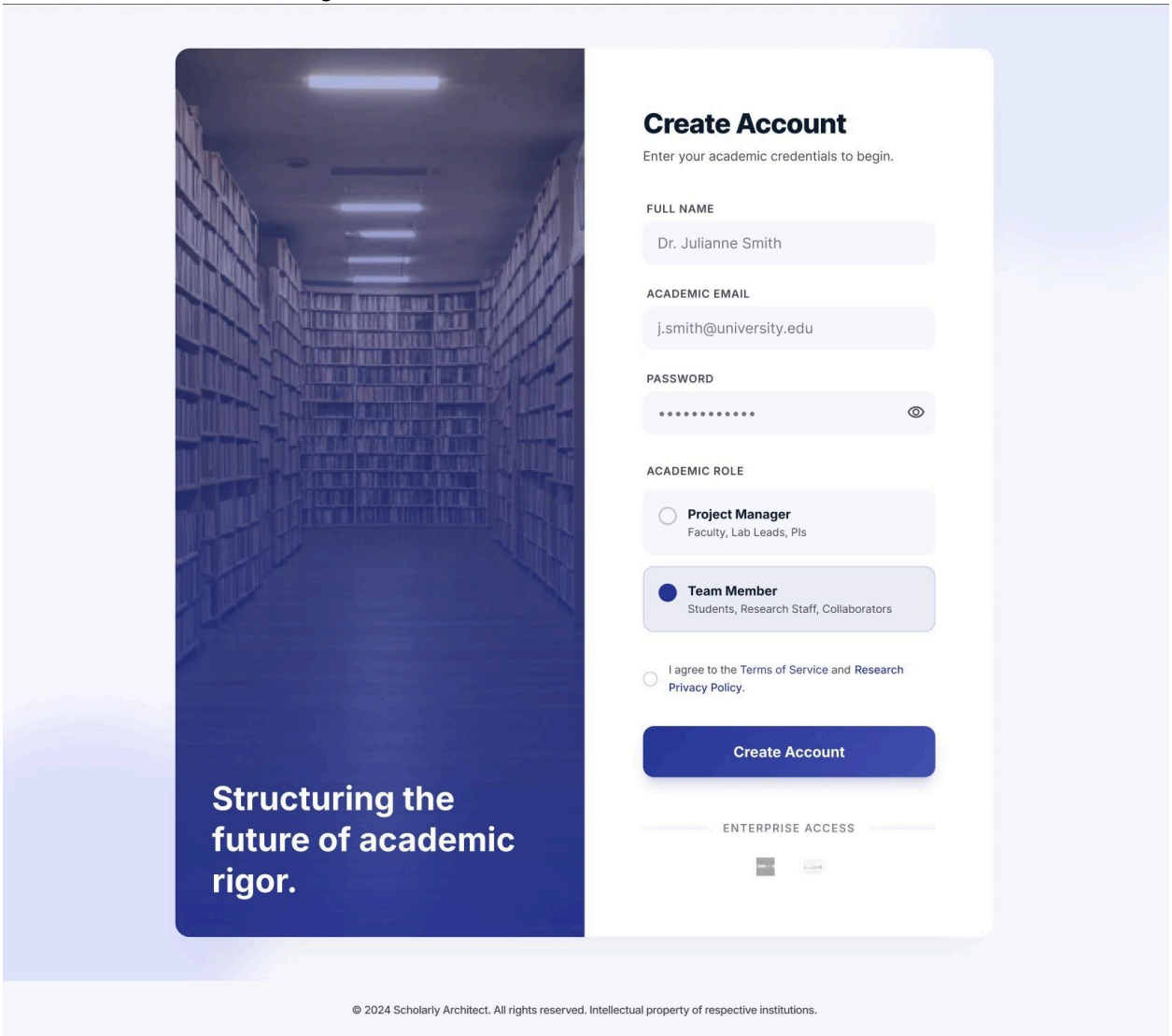
The main user interface requirements are as follows:

1) The system shall provide a responsive interface that can be accessed through desktop and mobile browsers.
2) The system shall maintain a consistent visual structure, navigation menu, and page layout across all major modules.
3) The system shall provide clear action buttons such as Sign In, Create Account, Create Project, Create Task, Invite Team Member, Update Progress, Save Profile, Cancel, and Export Report where appropriate.

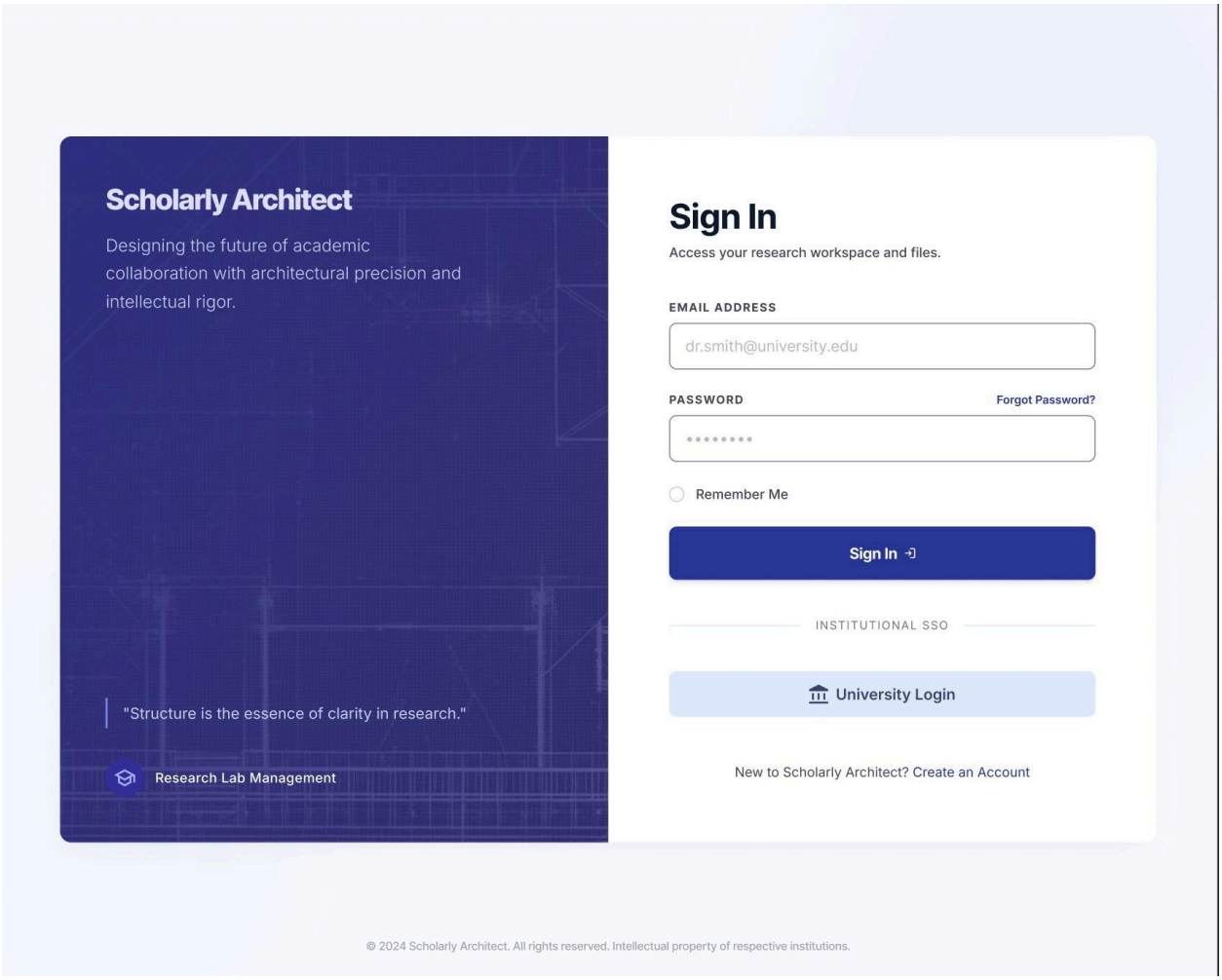
4) The system shall display task and project information using clear labels and status indicators such as Drafting, Active, In Progress, Done, Healthy, At Risk, High Priority, Medium, and Low.
5) The system shall provide input forms for registration, login, task creation, project creation, file upload, and profile update.
6) The system shall display validation or error messages for invalid login credentials, incomplete required fields, invalid invitation codes, unsupported uploads, or unauthorized actions.
7) Search fields shall be available in relevant modules to help users find projects, tasks, files, or team-related information more efficiently.

Detailed visual design, styling, and screen composition are documented separately in the approved Figma prototype used as the UI design reference for this chapter.

1. Create Account Page



2. Login Page



3.2 Hardware Interfaces

Collabra does not require any specialized or dedicated hardware devices. As a standalone web-based platform, the system interacts mainly with standard user devices and hosting infrastructure. This is consistent with the operating environment described in Chapter 2.

The supported hardware interface includes:

1) Desktop and laptop computers, which serve as the primary devices for project managers and team members to access the web application.
2) Standard input/output devices such as keyboard, mouse, touchpad, touchscreen, and monitor.
3) Cloud-hosted server infrastructure used to run the application logic, handle requests, and maintain data availability.
4) Persistent storage infrastructure used for saving user-uploaded files, project data, and task-related records.

The interaction between the software and hardware is limited to normal web application usage. Users provide inputs through browser-supported devices, while outputs are presented visually on screen. File uploads are transferred from the user's local device to the system's storage

environment. No direct communication with external hardware sensors, scanners, or embedded devices is required in this version of the system.

3.3 Software Interfaces

Collabra is designed as a standalone web-based application, therefore its software interfaces are limited to internal application components and a small number of supporting services. Chapter 2 explicitly states that the current version does not include integration with external productivity tools, enterprise systems, or communication platforms.

The main software interfaces are as follows:

1) **Web Browser Interface**
   The system shall be accessible through standard modern web browsers such as Google Chrome, Microsoft Edge, Firefox, and Safari. The browser functions as the client interface through which users access all major features of the system.

2) **Application Server Interface**
   The application layer handles authentication, authorization, project management, task processing, dashboard generation, collaboration logic, and reporting functions. Chapter 2 states that the system may be implemented using modern web technologies such as React.js on the frontend and Node.js or Python on the backend.

3) **Database Interface**
   The system shall interface with a Relational Database Management System (RDBMS) to store and manage user accounts, project data, tasks, team information, comments, progress records, reports, and file metadata. This supports the requirement to maintain a centralized “Single Source of Truth” for project collaboration.

4) **File Storage Interface**
   The system shall interface with local or server-based file storage for uploaded project files and supporting documents. This interface is required to support file sharing and collaboration features within project and task contexts.

5) **Email Service Interface**
   The system depends on the availability of third-party SMTP servers for invitation delivery and related communication support, as stated in the assumptions and dependencies section.

The primary data exchanged across these software interfaces includes:

- user registration and login data
- project records and project membership data
- Task details, deadlines, and priorities
- progress updates and comments
- uploaded file metadata and storage references
- dashboard summaries and reporting data

3.4 Communications Interfaces

Collabra requires network-based communication between users and the application server because it operates as a web-based platform. The primary communication interface is the interaction between the client browser and the server over internet protocols.

The main communication requirements are as follows:

1) The system shall use **HTTP/HTTPS** for communication between the client browser and the web server.
2) The system shall use **HTTPS** as the preferred secure communication standard for transmitting login credentials, profile data, project information, task updates, comments, and uploaded files.
3) The system shall support **form-based communication** for account registration, login, project creation, task creation, profile updates, project joining, and progress updates.
4) The system shall support **file transfer communication** for uploading and retrieving project-related documents.
5) The system shall support **SMTP-based email communication** for project invitations and other notification-related messages where required.
6) The system shall provide timely data updates for dashboard information, task boards, project progress, and activity feeds, assuming users have stable internet access.

In terms of communication security, the system must follow the constraints in Chapter 2 by ensuring password protection through encryption and by using secure file-sharing practices. The Figma design also reflects this requirement by showing secure access, institutional login, and encrypted file repository messaging in several screens.

4. System Features

<This template illustrates organizing the functional requirements for the product by system features, the major services provided by the product. You may prefer to organize this section by use case, mode of operation, user class, object class, functional hierarchy, or combinations of these, whatever makes the most logical sense for your product.>

4.1 System Feature 1

<Don't really say "System Feature 1." State the feature name in just a few words.>

4.1.1 Description and Priority

<Provide a short description of the feature and indicate whether it is of High, Medium, or Low priority. You could also include specific priority component ratings, such as benefit, penalty, cost, and risk (each rated on a relative scale from a low of 1 to a high of 9).>

4.1.2 Stimulus/Response Sequences

<List the sequences of user actions and system responses that stimulate the behavior defined for this feature. These will correspond to the dialog elements associated with use cases.>

4.1.3 Functional Requirements

<Itemize the detailed functional requirements associated with this feature. These are the software capabilities that must be present in order for the user to carry out the services provided by the feature, or to execute the use case. Include how the product should respond to anticipated error conditions or invalid inputs. Requirements should be concise, complete, unambiguous, verifiable, and necessary. Use "TBD" as a placeholder to indicate when necessary information is not yet available.>

<Each requirement should be uniquely identified with a sequence number or a meaningful tag of some kind.>

REQ-1:
REQ-2:

4.2 System Feature 2 (and so on)

5. Other Nonfunctional Requirements

5.1 Performance Requirements

<If there are performance requirements for the product under various circumstances, state them here and explain their rationale, to help the developers understand the intent and make suitable design choices. Specify the timing relationships for real time systems. Make such requirements as specific as possible. You may need to state performance requirements for individual functional requirements or features.>

5.2 Safety Requirements

<Specify those requirements that are concerned with possible loss, damage, or harm that could result from the use of the product. Define any safeguards or actions that must be taken, as well as actions that must be prevented. Refer to any external policies or regulations that state safety issues that affect the product's design or use. Define any safety certifications that must be satisfied.>

5.3 Security Requirements

<Specify any requirements regarding security or privacy issues surrounding use of the product or protection of the data used or created by the product. Define any user identity authentication requirements. Refer to any external policies or regulations containing security issues that affect the product. Define any security or privacy certifications that must be satisfied.>

5.4 Software Quality Attributes

<Specify any additional quality characteristics for the product that will be important to either the customers or the developers. Some to consider are: adaptability, availability, correctness, flexibility, interoperability, maintainability, portability, reliability, reusability, robustness, testability, and usability.

Write these to be specific, quantitative, and verifiable when possible. At the least, clarify the relative preferences for various attributes, such as ease of use over ease of learning.>

5.5 Business Rules

<List any operating principles about the product, such as which individuals or roles can perform which functions under specific circumstances. These are not functional requirements in themselves, but they may imply certain functional requirements to enforce the rules.>

6. Other Requirements

<Define any other requirements not covered elsewhere in the SRS. This might include database requirements, internationalization requirements, legal requirements, reuse objectives for the project, and so on. Add any new sections that are pertinent to the project.>

Appendix A: Glossary

<Define all the terms necessary to properly interpret the SRS, including acronyms and abbreviations. You may wish to build a separate glossary that spans multiple projects or the entire organization, and just include terms specific to a single project in each SRS.>

Appendix B: Analysis Models

<Optionally, include any pertinent analysis models, such as data flow diagrams, class diagrams, state-transition diagrams, or entity-relationship diagrams.>

Appendix C: To Be Determined List

<Collect a numbered list of the TBD (to be determined) references that remain in the SRS so they can be tracked to closure.>